From Features to Machine Learning

Week 12 — Turning image features into a classifier

Practical Image Processing

Lecture 12 ▪ 60–70 min

A turning point in the course

- Weeks 1–11: *we* designed every step (filters, edges, corners, warps).
- Now: we let the computer *learn* the decision from labelled examples.
- The bridge: turn an image into a **feature vector**, then train a **classifier**.

Today's real task

Classify Iranian car models (206 / 405 / Dena) from **HOG** features — using your `E:/data` images.

Roadmap

The feature $\rightarrow$ classifier pipeline

Classifiers

Results & overfitting

In-class exercise

Deep dive

Wrap-up

The feature → classifier pipeline

The classic recognition pipeline

1. **Input image** $\rightarrow$
2. **Feature extraction** (turn pixels into a compact vector) $\rightarrow$
3. **Classifier** (a learned function vector $\rightarrow$ label) $\rightarrow$
4. **Prediction** (206 / 405 / Dena).

Key idea

Until $\sim$2012, essentially all recognition worked this way. The features were hand-designed; only the classifier was learned.

Why not feed raw pixels?

- A 96×96 image is 9216 numbers — huge, and every shift/lighting change moves them all.
- Raw pixels are not **invariant**: the same car, moved 5 px, looks "different".
- **Features** (HOG, colour histograms, ORB) compress the image into a stable, meaningful vector.

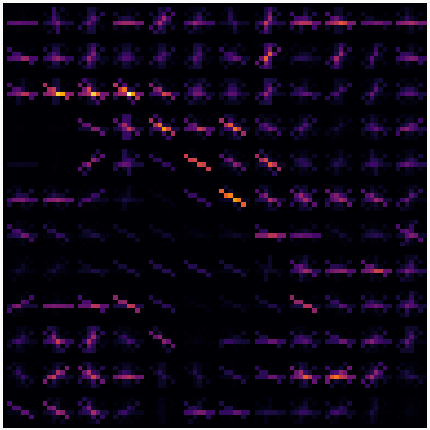
Math

Good features make classes **separable**: similar things land near each other in feature space.

HOG: Histogram of Oriented Gradients



a real 206



its HOG (edge-orientation) map

HOG divides the image into cells and, in each, histograms the gradient directions — a compact shape signature.

Why HOG works for shape

- Cars differ in *shape* (roofline, lights, grille) — captured by gradient directions.
- Histogramming per cell gives tolerance to small shifts and lighting changes.
- HOG + linear SVM was the state of the art for pedestrian/vehicle detection for years.

Key idea

HOG throws away exact pixel values and keeps *where edges point* — the essence of shape.

Classifiers

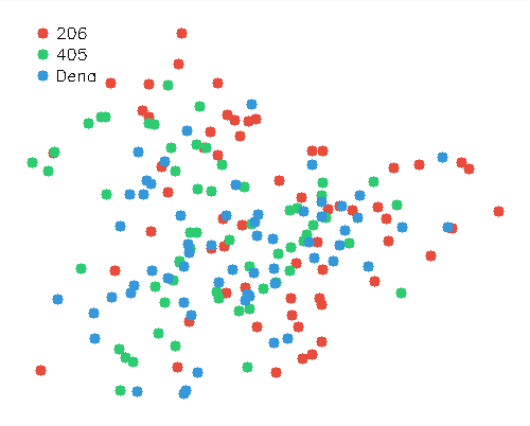
What a classifier learns

- Given feature vectors with labels, it learns a **decision boundary** between classes.
- **k-NN**: label a point by the majority vote of its k nearest training points.
- **SVM**: find the boundary with the widest **margin** between classes.

Math

k-NN memorizes; SVM optimizes a margin. Both need features where classes are separable.

Our classes in feature space (PCA to 2D)



- Each dot is a real car's HOG vector, projected to 2D with PCA.
- Classes overlap a lot — these three models genuinely look similar. Hard problem!

Train / test split — the golden rule

- Split data: **train** the classifier on one part, **test** on unseen data.
- Testing on training data lies — the model can just memorize.
- We used 70% train / 30% test, **stratified** (each class balanced in both).

✕ Pitfall

Never report accuracy on data the model has seen. That number is meaningless for real use.

Results & overfitting

Our real results (and an honest lesson)

Classifier	Train acc	Test acc
k-NN (k=5)	0.65	0.43
SVM (RBF)	1.00	0.48

Chance is 0.33 (3 classes). Both beat chance — but the SVM scores **100% on train, 48% on test**. That gap is **overfitting**.

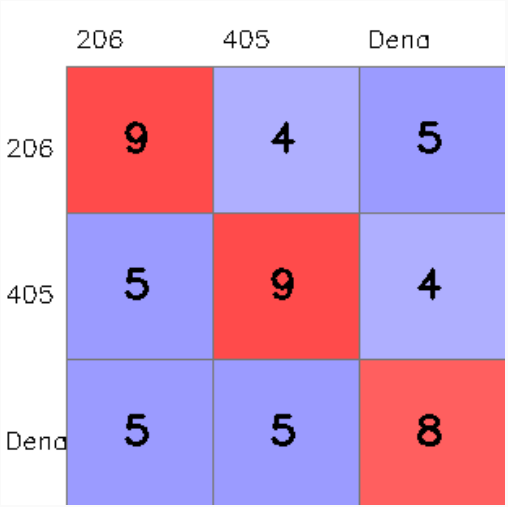
Overfitting, explained

- **Overfitting**: the model memorizes training quirks instead of the real pattern.
- Symptom: train accuracy $\gg$ test accuracy (here 1.00 vs 0.48).
- Cures: more data, simpler model, regularization, better/invariant features.

Key idea

A model that is perfect on training data and mediocre on test data has learned the wrong thing.

Reading the confusion matrix



- Rows = true class, columns = predicted.
- Diagonal = correct; off-diagonal = the mistakes.
- Shows *which* models get confused — far more useful than a single accuracy number.

Metrics beyond accuracy

- **Accuracy** hides imbalance: 95% "all normal" is useless if 5% are the faults you need.
- **Precision**: of predicted-positive, how many were right? **Recall**: of actual-positive, how many did we catch?
- **F1**: their harmonic mean. Report these per class, not one global number.

Math

$$\text{Precision} = \frac{TP}{TP + FP}, \quad \text{Recall} = \frac{TP}{TP + FN}, \quad \text{F1} = \frac{2\,PR}{P + R}.$$

In-class exercise

In-class exercise (10 min)

Diagnose the model

1. Our SVM got train 1.00, test 0.48. Name the problem and two ways to reduce it.
2. A fault detector reports 96% accuracy but faults are 4% of data. Why might it be useless?
3. You want to tell 206 from 405. What feature besides HOG might help separate them?

Debrief

- (1) **Overfitting**: gather more data, simplify/regularize the SVM (lower C), or use better features.
- (2) Predicting "no fault" always scores 96% but has **zero recall** on faults — accuracy is the wrong metric.
- (3) **Colour histograms** (models differ in typical colour), size/aspect, or logo region features.

Key idea

A number without a train/test split and a confusion matrix tells you almost nothing.

Deep dive

Feature engineering vs feature learning

- Today we **engineered** features (HOG) and learned only the classifier.
- The 48% ceiling on look-alike cars hints at the limit of hand-designed features.
- **Deep learning** (Weeks 13–14) *learns the features too*, directly from pixels.

Key idea

The classical pipeline plateaus on hard, fine-grained problems. That plateau is exactly why CNNs took over.

Why our SVM overfit so hard

- Only 126 training vectors, each 900-dimensional — far more features than examples.
- An RBF SVM with high C can bend the boundary to fit every training point exactly.
- Result: train 1.00, but the boundary is tuned to noise $\rightarrow$ test collapses to 0.48.

Key idea

More dimensions than samples is a red flag: the model has enough freedom to memorize. Reduce C, reduce dimensions (PCA), or get more data.

Cross-validation & not fooling yourself

- A single split is noisy. **k-fold cross-validation** averages over k splits for a stable estimate.
- Tune hyper-parameters on a **validation** set (or CV) — never on the test set.
- The test set is used *once*, at the very end. Peeking at it inflates your reported score.

Where this fits DocScan

- DocScan must read digits. A digit is a small image; a classifier maps it to 0–9.
- Today's pipeline (features $\rightarrow$ classifier) is exactly how classic OCR worked.
- Next week we replace hand features with a **CNN** that learns them — and accuracy jumps.

The whole ML pipeline in code

```
X = [hog(gray(img)) for img in images]      # features
Xtr, Xte, ytr, yte = train_test_split(X, y, test_size=0.3,
                                      stratify=y)
clf = SVC(kernel='rbf', C=10).fit(Xtr, ytr) # learn
print(accuracy_score(yte, clf.predict(Xte)))# HONEST score
print(confusion_matrix(yte, clf.predict(Xte)))
```

Extract features $\rightarrow$ split $\rightarrow$ fit $\rightarrow$ evaluate on unseen data. Always in that order.

Wrap-up

What you should now be able to do

- Build a features $\rightarrow$ classifier pipeline (HOG + SVM/k-NN) on real images.
- Split train/test correctly and read a confusion matrix.
- Recognize overfitting (train $\gg$ test) and name cures.
- Explain why accuracy alone misleads, and when precision/recall matter.

This week's deliverable

Due before Week 13 (see `project/`)

1. Extract HOG features for 2–3 classes from `E:/data`; build X, y.
2. Train k-NN and SVM with a stratified train/test split; report both accuracies.
3. Show the confusion matrix; identify which classes get confused.
4. Diagnose overfitting and try one cure; report whether the test score improved.

Features make it learnable; the split keeps you honest.

Hand-designed features hit a ceiling — next week we learn the features themselves.